

Computational Methods in Particle Physics

Part 5: Data & Statistical analysis

Drs Ben Hodkinson & Holly Pacey
Oxford HEP DPhil Programme
Hilary Term 2025

Part 1) Fitting using RooStats

Recommended reading

Wouter Verkerke's lecture on [Systematic Uncertainties and profiling](#)
→ *These slides are heavily based on parts of this*

Statistical models

$$f(\mathbf{x} \mid \phi) = f(\mathbf{x} \mid \mu, \theta).$$

- HEP data is inherently random
 - QFT is a probabilistic theory
 - Classical randomness impacts detector response
 - Theoretical calculations and detector modelling are imperfect
- How can a bona fide discovery be distinguished from a statistical fluctuation or mis-modelling?
- A statistical model of your data is required to reflect these uncertainties
- Probability distribution function (PDF) of the event rates $\mathbf{x}$ given parameters ϕ
 - The observed data is one value of $\mathbf{x}$ drawn from the PDF
- Model parameters include parameters of interest (POIs) μ and nuisance parameters (NPs) θ
 - POI = Cross-section, SM quantity, signal strength, ...
 - NPs = random variables that affect event rates but with unknown true values
 - Background normalisation factors
 - Systematic uncertainties → quantify our incomplete knowledge of the statistical model

Building a statistical model

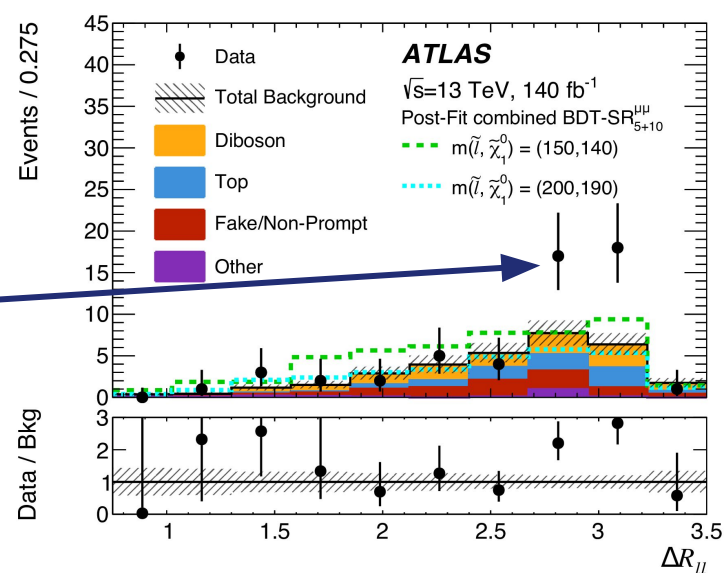
Example: Collider search

What is the likelihood of measuring these data counts in a binned distribution, given the background model?

- The PDF is built from products of Poisson distributions

$$\text{Pois}(n; \nu) = \frac{\nu^n e^{-\nu}}{n!}$$

- The expected event counts ν are subject to deviations due to things we don't know about the data $\rightarrow$ nuisance parameters



Observed event counts

Nuisance parameters

Auxiliary measurement
(eg. in a control region)

$$f(\mathbf{x} \mid \boldsymbol{\phi}) = f(\mathbf{n}, \mathbf{a} \mid \boldsymbol{\eta}, \boldsymbol{\chi}) = \prod_{c \in \text{channels}} \prod_{b \in \text{bins}} \text{Pois}(n_{cb} \mid \nu_{cb}(\boldsymbol{\eta}, \boldsymbol{\chi})) \prod_{\chi \in \boldsymbol{\chi}} c_{\chi}(a_{\chi} \mid \chi)$$

Expected event counts (based on MC simulation or data-driven estimate)

Constraint term to model impact of NPs (often Gaussian)

Building a statistical model

Example: Continuous sig/bg functional forms.
Data still binned as a poisson counting exp.

$$s_i = s_{\text{tot}} \int_{\text{bin } i} f_s(x; \vec{\theta}_s) dx ,$$

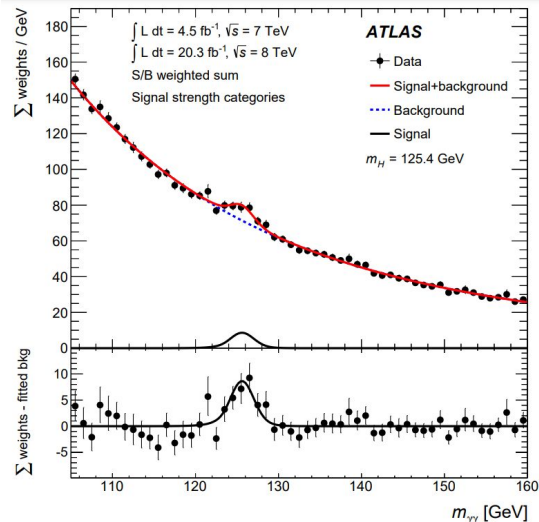
$$b_i = b_{\text{tot}} \int_{\text{bin } i} f_b(x; \vec{\theta}_b) dx .$$

$$\mathcal{L}(\mu_{\text{sig}}) \supset \prod_{j=1}^N \text{Pois}(N_j^{\text{Obs}} | \mu_{\text{sig}} s_j + b_j) \times f_s \times f_b$$

Expected event counts in bin j
Evaluated via function
PDFs * total exp. yield.

Signal strength
parameter of
interest

PDFs represent the sig and bg
functions



Observed
event counts

Likelihood

The **likelihood** tells us how plausible the data is given the parameters of the model.

It is constructed by fixing the event counts in the statistical model to the observed data.

$$L(\phi) = f(\mathbf{x} = \mathbf{x}_{\text{obs}} \mid \phi).$$

For example, if we believe our data is distributed as a Gaussian, and we want to know the mean μ and standard deviation σ , we would use the following likelihood:

$$p(d|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \prod_i^N \exp \left(\frac{-(d_i - \mu)^2}{2\sigma^2} \right)$$

The likelihood is small if the data is far away from the distribution suggested by the model

Likelihood

$$\mathcal{L}(\mu, \theta) = \mathcal{L}_{\text{data}}(N|\mu, \theta) \cdot \mathcal{A}(\tilde{\theta}|\theta)$$

$\mathcal{L}_{\text{data}}$	The portion of the likelihood representing the data “N” ➤ For binned distributions, $\mathcal{L}_{\text{data}} = \prod_i \text{PoiSS}(N_i E_i(\mu, \theta))$
$\mathcal{A}$	Auxiliary information (measurements) • Eg. detector resolution, lepton efficiency • Gaussian or Poissonian
N	Number of observed events (after analysis selections)
μ	Parameters of interest (POI) that we want to measure • Eg. Cross-section, SM parameter, etc.
θ	Set of nuisance parameters (NP) corresponding to the auxiliary measurements. • This represents new measurements in the phase space of the data • Used to implement systematic uncertainties or background normalisations
$\tilde{\theta}$	Set of old/auxiliary measurements (and is usually fixed, e.g. relates to what the jet experts tell is the 1-sigma jet-energy scale measurement uncertainty)

Fitting the model parameters

- The model parameters ϕ are determined by finding the values ϕ' that maximise the likelihood
- To simplify this calculation, usually the the negative log-likelihood is minimised
 - For typical Gaussian forms this converts products of exponentials to sums of parabolic terms
→ simpler optimisation

$$\lambda = -2 \log L(\phi)$$

- There's no guarantee this gives the 'true' parameter values (as there may be fluctuations in the data), but it does provide a sensible estimate.

Test Statistics

A **test statistic** is any function of the data, which has an obtainable probability distribution.

The Neyman-Pearson lemma states that, in the absence of systematics, the ratio of likelihoods between two hypotheses is the most powerful statistical test to separate those hypotheses: $\lambda(x) = \mathcal{L}(x|H_1)/\mathcal{L}(x|H_2)$

Doesn't hold with systematics but motivation for profile likelihood ratio $\lambda(\mu) = \mathcal{L}(\mu, \hat{\theta}_\mu)/\mathcal{L}(\hat{\mu}, \hat{\theta})$

The numerator is the likelihood maximised at a fixed point of the POI (conditional likelihood) and the denominator is the likelihood maximised over all parameters (unconditional likelihood).

The one-sided profile likelihood test statistic is $t_\mu = -2 \ln \lambda(\mu)$

Sampling distributions

$$t_\mu = -2 \ln \lambda(\mu)$$

The sampling distribution of a test statistic is simply its probability distribution under pseudo-experiments, i.e. under Poisson variations of N observed in each bin of your distribution

Wilk's theorem states that, for a single POI μ , the distribution of t_μ asymptotically approaches χ^2 (when N is large) with one degree of freedom.

$$f(t_\mu | \mu) = \frac{1}{\sqrt{2\pi}} \frac{1}{\sqrt{t_\mu}} e^{-t_\mu/2}$$

This yields (after integrating) the probability for an observed value of t_μ

$$p_\mu = 1 - F(t_\mu | \mu) = 2(1 - \Phi(\sqrt{t_\mu}))$$

→ use to perform **hypothesis test** or produce **confidence interval**

Probability models and likelihoods in practice

Three steps to running your fits:

- 1) **Construction of probability models and likelihood functions**
 - Implementation of systematic uncertainties typically leads to very complex models
- 2) **Minimization of the likelihood functions**
 - For parameter estimation, variance estimate, likelihood-ratio intervals
 - HEP standard is MINUIT
- 3) **Construction of test statistics and calculation of their distributions**
 - p-values, confidence intervals, Bayesian intervals, Bayes factors

Fitting distributions in practice

- ROOT ecosystem
 - [RooFit](#) – language for building probability models of events in a physics analysis. Comprises datasets, likelihoods, minimization, toy data generation, visualization and persistence
 - [HistFactory](#) – tool for constructing binned likelihood template models (a specific type of statistical model)
 - [RooStats](#) – suite of statistical tests operating on RooFit probability models
- Python ecosystem
 - [Cabinetry](#) – python library for building HistFactory statistical models (for binned likelihood models)
 - [PyHF](#) – pure-python implementation of the HistFactory statistical model for multi-bin histogram-based analysis (completed independent of ROOT)
 - [ZFit](#) - similarly python based, similar to RooFit, binned/unbinned fits

RooFit

- RooFit is an object-oriented modelling framework written in C++
- It is built on top of ROOT (and is built by default in binary distributions)
- Heavily uses abstraction to be able to create high level models, which can later be built upon by even higher level packages (RooStats, HistFactory, etc) that implement concrete statistical tools.
- RooFit is very flexible and powerful, but has a learning curve to use more complex setups.

Fitting distributions with RooFit

- The **RooFit** library provides a toolkit for modeling the expected distribution of events in a physics analysis.
- Models can be used to perform likelihood fits, produce plots, and generate "toy Monte Carlo" samples for various studies.
- The core functionality of RooFit is to **enable the modeling of counting/event data** distributions, where each event is a discrete occurrence in time, and has one or more measured observables associated with it.
- Experiments of this nature result in **datasets obeying Poisson (or binomial) statistics**. The modeling language for such distributions are probability density functions $F(x;p)$ that describe the probability density of observables x in terms of function in parameter p .

RooFit's Modular Approach

- Mathematical objects are represented as C++ objects
- A feature of this design philosophy is that all RooFit models always consist of multiple objects.

<i>Math concept</i>	<i>Math symbol</i>	<i>RooFit (base)class</i>
Variable	x	RooRealVar
Function	$f(x)$	RooAbsReal
P.D.F.	$F(x; p)$	RooAbsPdf
Integral	$\int_{x_{min}}^{x_{max}} f(x) dx$	RooRealIntegral
Space point	$\vec{x}$	RooArgSet
Addition	$fF(x) + (1 - f)G(x)$	RooAddPdf
Convolution	$f(x) \otimes g(x)$	RooFFTConvPdf

Table 1 - Correspondence between selected math concepts and RooFit classes

PDFs in RooFit

The following basic shapes are provided as p.d.f.s

Shape	Class	Description
Gaussian	RooGaussian	Normal distribution
Bifurcated Gaussian	RooBifurGauss	Variation of Gaussian with different widths on the low and high side of the mean
Exponential	RooExponential	Exponential decay distribution
Polynomial	RooPolynomial	Polynomial shapes with coefficients for each power of x
Chebyshev polynomial	RooChebyshev	Chebyshev polynomials of the first kind
Poisson	RooPoisson	Standard Poisson distribution

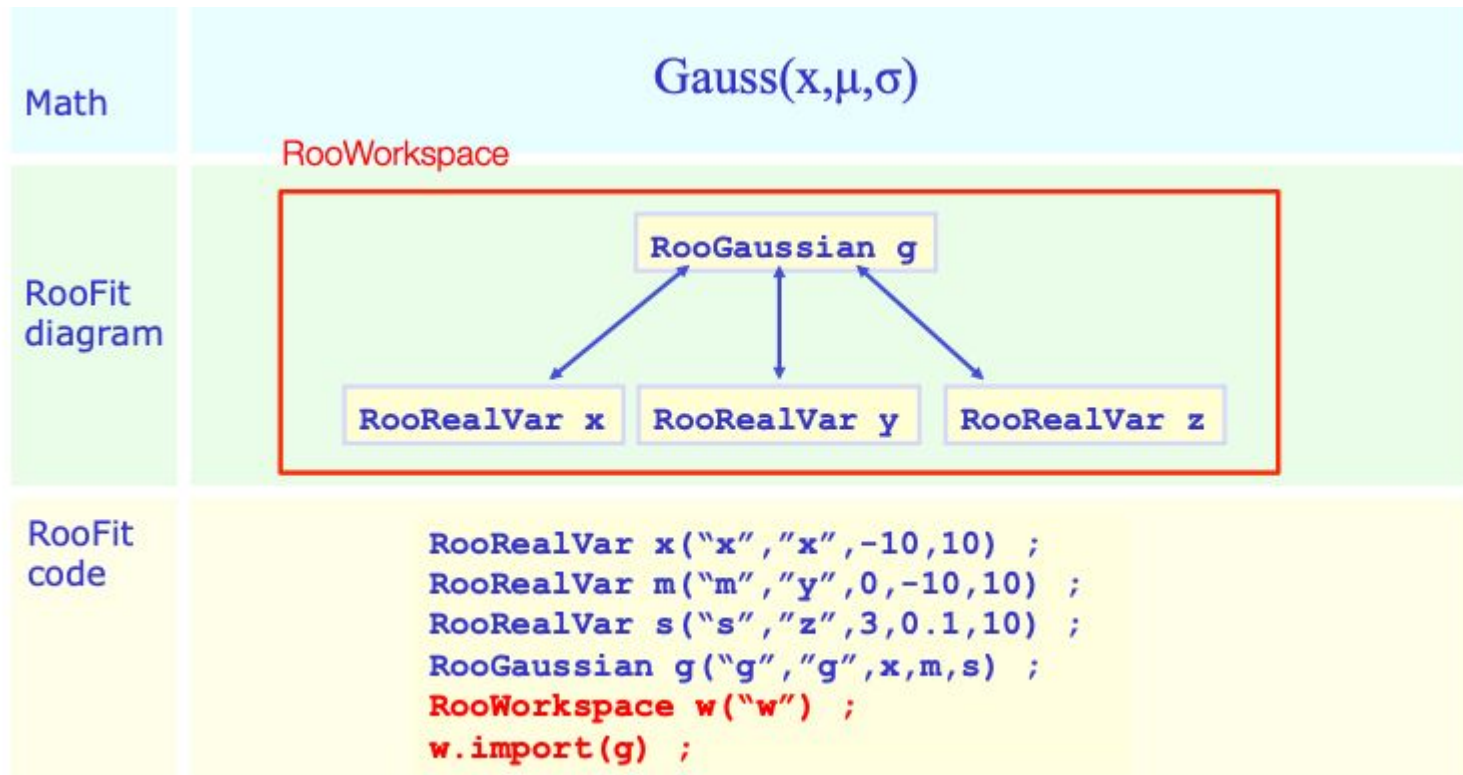
PDFs in RooFit

In addition to the basic shapes RooFit also implements a series of shapes that are commonly used to model physical 'signal' distributions

Shape	Class	Description
Landau	RooLandau	This function parameterizes energy loss in material and has no analytical form. RooFit uses the parameterized implementation in TMath::Landau.
Breit-Wigner	RooBreitWigner	Models resonance shapes
Voigtian	RooVoigtian	Breit-Wigner convolved with a Gaussian – models resonance given finite detector resolution
Crystal ball	RooCBShape	Gaussian with a tail on the low side – describes the effect of radiative energy loss in an invariant mass
Novosibirsk	RooNovosibirsk	Asymmetric Gaussian with long tail on one side and short tail on the other
Argus	RooArgusBG	Empirical formula to model the phase space of multi-body decays near threshold – use in B physics

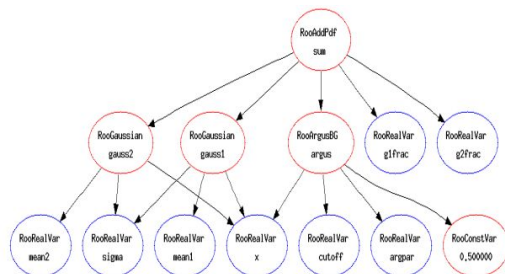
Workspace

The workspace serves a container class for all objects created

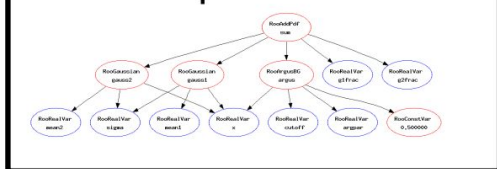


Workspace

- Factorizes the process of building and using likelihood functions
- Makes it very easy to share the likelihood of a (potentially very complex) physics analysis in a way that is easy to inspect, modify, and use.
 - Can upload these to [HEPData](https://hepdata.net) for analysis preservation and re-interpretation.



RooWorkspace



model.root



```
RooWorkspace w("w") ;  
w.import(sum) ;  
w.writeToFile("model.root") ;
```

```
// Resurrect model and data  
TFile f("model.root") ;  
RooWorkspace* w = f.Get("w") ;  
RooAbsPdf* model = w->pdf("sum") ;  
RooAbsData* data = w->data("xxx") ;
```

```
// Use model and data  
model->fitTo(*data) ;
```

```
RooPlot* frame =  
    w->var("dt")->frame() ;  
data->plotOn(frame) ;  
model->plotOn(frame) ;
```

Wouter Verkerke, NIK-HEF

A complete ecosystem

HistFactory and **RooStats** are built on top of RooFit

HistFactory is a class-based package that builds a likelihood and data out of input histograms, including systematics, which imported into a **RooWorkspace** and written to a **TFile**.

RooStats is a collection of tools used to perform statistical calculations

ROOT tutorials (`$ROOTSYS/tutorials/roofit`) contain not only examples for RooFit, HistFactory, and RooStats, but also a nice set of top-level tools you can use out-of-the-box (or with minimal tuning) for your analysis needs.

HistFactory – structured building of binned template models

- LHC analyses require modelling 10s or 100s of systematic uncertainties in $\mathcal{O}(10)$ samples and multiple channels (in some cases $\mathcal{O}(100)$).
- HistFactory provides a structured formalism to piece together such binned models from specifications.
 - Channels
 - Samples
 - POI
 - Luminosity
 - Systematics

Model building with HistFactory

```
// external input in form of TH1 shown in green
```

```
// Declare ingredients of measurement
```

```
HistFactory::Data data ;  
data.SetHisto(data_hist) ;
```

```
HistFactory::Sample signal("signal") ;  
signal.SetHisto(sample_hist) ;
```

```
HistFactory::Sample qcd("QCD") ;  
qcd.SetHisto(sample_hist) ;
```

```
HistFactory::HistoSys hsys("QCD_JetEnergyScale") ;  
hsys.SetHistoLow(sample_hist_sysdn) ;  
hsys.SetHistoHigh(sample_hist_sysup) ;  
qcd.AddHistoSys(hsys) ;
```

```
HistFactory::Channel channel("VBF") ;  
channel.SetData(data) ;  
channel.AddSample(sample) ;
```

```
HistFactory::Measurement meas("MyAnalysis") ;  
meas.AddChannel(channel) ;
```

```
// Now build RooFit model according to specs
```

```
HistFactory::HistoToWorkspaceFactoryFast h2w(meas) ;
```

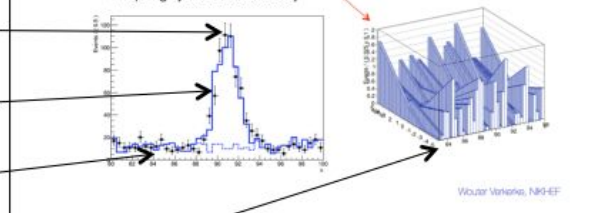
```
RooWorkspace* w = h2w.MakeCombinedModel(meas) ;
```

```
w->Print("t") ;
```

```
w->writeToFile("test.root") ;
```

Example of model building with HistFactory

- An example of model building with HistFactory
- Measurement consists of one channel ("VBF")
- The VBF channel comprises
 1. A data sample
 2. A template model of two samples ("signal" and "qcd")
 3. The background sample has a "JES" template morphing systematic uncertainty



Warning: .ipynb notebooks not listed here may crash pplxint...

Getting Started with RooFit

RooFit Basics: [RooFitBasics.ipynb](#) → Create and fit a Gaussian PDF

RooFit Composite model: [rf201_composite.py](#) → Combining multiple PDFs

Number counting example:

- Setup: [NumberCounting_Step1_Setup.py](#)
- Fit: [NumberCounting_Step2_Fit.py](#)
- Likelihood Scan: [NumberCounting_Step3_LikelihoodScan.py](#)

More examples available in the [ROOT:Fit Tutorials](#)

Full Higgs search example: [part2_discoHiggs.py](#)

Fitting with PyHF

Hands-on

If you're interested in running binned likelihood fits in the python ecosystem, you can work through the PyHF tutorial

<https://pyhf.github.io/pyhf-tutorial/introduction.html>

Note this is geared towards ATLAS-style binned likelihood fits



Get started

Introduction to HistFactory Models

Introduction to Workspaces

Using HEPData

Common use and applications

Making a Pull Plot

Playing with Toys

Performing a Combination

Model-Independent Upper Limits

Fitting with ZFit

Hands-on

If you're interested in running unbinned/binned likelihood fits in the python ecosystem, you can work through the ZFit tutorial

<https://zfit-tutorials.readthedocs.io/en/latest/>

Note this is geared towards LHCb-style likelihood fit



[Introduction](#)

[Quickstart with zfit](#)

[Introduction](#)

[What's new](#)

[Components](#)

[05 FitResult](#)

[13 Kernel Density Estimation](#)

[20 Composite Models](#)

[30 Binned Models](#)

[50 Custom code and different run modes](#)

[60 Custom PDF](#)

[62 Multidimensional custom PDF](#)

[77 - Custom Minimizers](#)

[80 Toy Study](#)

[90 Serialization](#)

[Guides](#)

[Custom model guide](#)

[Constraints, simultaneous fits, discovery and sPlot](#)

[HPC with TensorFlow](#)

[HPC in Python](#)

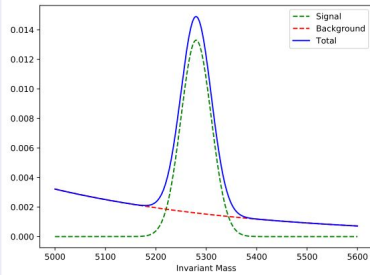
[Statistics with TensorFlow](#)

Part 2) Background Subtraction with SWeights

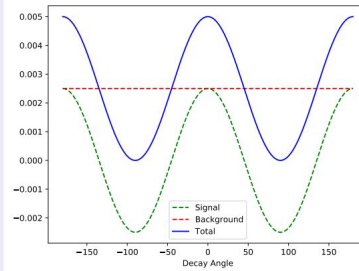
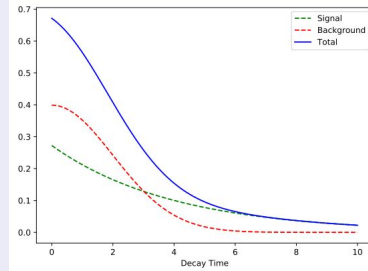
Background Subtraction

- A key component of many HEP results, e.g. in LHCb analyses.
- Typically we have a **targeted control** variable we want to measure (e.g. a particle lifetime, where the signal/background are hard to separate)
And another **discriminating** variable (e.g. a mass spectrum, where the signal has a clear peak from bg continuum)

“Discriminant variable”



“Control variable(s)”



Background subtraction options

How to subtract the background from our control variable?

1. Fit both distributions together?
Need to have good modelling for both components.
2. Sidebands/slicing?
Requires binning so not necessarily optimal, and needs 2 variables to be **independent**.
3. Use the SPlot method: a statistical tool for unfolding a signal distribution.
This is done via “sWeights”: Derive s/b weights for each event to project out signal/background in the control variable in data. Basically per-event slicing. Still Needs 2 variables to be **independent**.

SWeights

- Given the signal and background models in the discriminating variable
- Fit to data.
- Derive the sWeight_s/b for each data point, via equations like that below.
- Apply the sWeight_s to each data event in the control variable -> retrieve the signal distribution only!

- For a total p.d.f. of the form

$$f(m, t) = \underbrace{zg_s(m)h_s(t)}_{\text{Signal}} + \underbrace{(1-z)g_b(m)h_b(t)}_{\text{Background}} \quad (1)$$

- Then can project signal and background out with functions (proof in backup),

$$w_s(m) = \underbrace{\frac{\alpha_s g_s(m) + \alpha_b g_b(m)}{g(m)}}_{\text{Signal}} \quad \text{and} \quad \underbrace{w_b(m) = \frac{\beta_s g_s(m) + \beta_b g_b(m)}{g(m)}}_{\text{Background}}, \quad (2)$$

by solving

$$\begin{pmatrix} W_{ss} & W_{sb} \\ W_{sb} & W_{bb} \end{pmatrix} \cdot \begin{pmatrix} \alpha_s & \beta_s \\ \alpha_b & \beta_b \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \text{where } W_{xy} = \int \frac{g_x(m)g_y(m)}{g(m)} dm. \quad (3)$$

Part 3) Bayesian Inference & Markov Chain Monte Carlo

Bayes' Theorem

LIKELIHOOD

The probability of "B" being True, given "A" is True

PRIOR

The probability "A" being True. This is the knowledge.

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

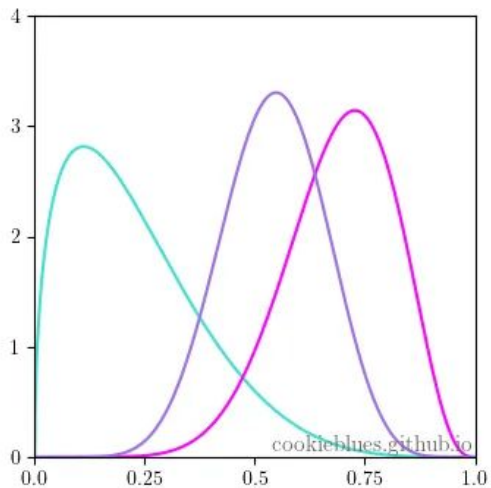
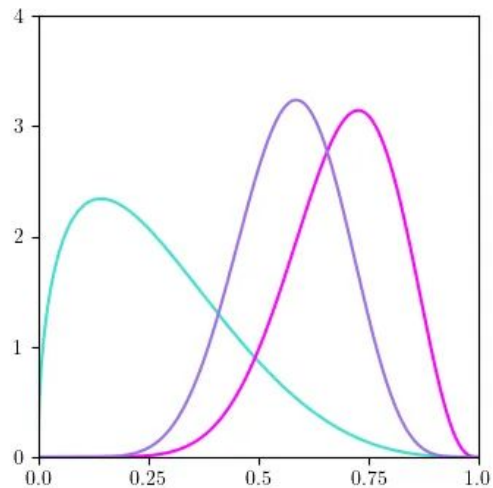
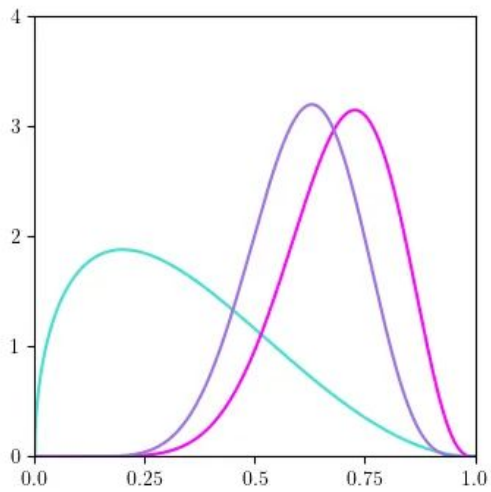
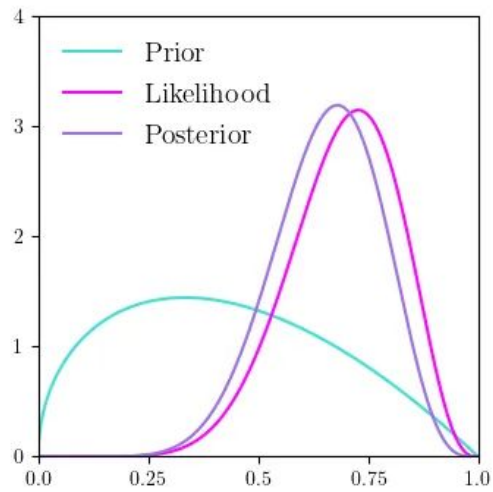
POSTERIOR

The probability of "A" being True, given "B" is True

MARGINALIZATION

The probability "B" being True.

For example, used to determine the (posterior) conditional probability of a hypothesis given the data $\rightarrow P(H | d)$



Examples of different priors applied to the same likelihood

Prior

A **prior** tells us the **degree of belief** we have in a hypothesis before seeing data.

Priors must be chosen carefully depending on the problem.

Priors have a *certain degree of subjectiveness*. But in many cases, such as particle physics, one uses reasonable values, such as the mass of a particle from the Particle Data Group (PDG) world average, which combined many published measurements.

Note that **when the data is informative, the posterior is insensitive** to prior distributions used; it is only **when we lack data that we must rely on prior beliefs**.

Finding the Posterior

In simple cases, such as the Gaussian case, it's possible to integrate and solve for the posterior.

Not only is this not always possible but it doesn't yield a generic solver.

It would be nice to have an approach that can be applied to a **high-dimensional** parameter space (many variables) *without being computationally prohibitive*.

Naively, one scans the parameter space and compute the likelihood at each point, multiplying by the prior, to solve for the posterior. **Computationally prohibitive**.

Markov Chain Monte Carlo (MCMC) solves this problem by **efficiently sampling from the posterior distribution**.

Markov Chain

Any algorithm that takes **only** the current state of the chain $\mathbf{X}^i$ to generate the next step $\mathbf{X}^{i+1}$ is a **Markov Chain** (i.e. it doesn't matter how the system arrived there)

$$p(X^{i+1}) = t(X^{i+1} | X^i)$$

Where $t(x|y)$ is the transition probability from one state to another.

In other words, the probability for a new state to be at a certain value depends on the conditional transition probability of the new state given the current state.

We can formulate a Markov Chain Monte Carlo (MCMC) algorithm that has the posterior from the Bayes formula as its equilibrium distribution.

Metropolis-Hastings

The **Metropolis-Hasting (MH)** algorithm is an MCMC method for sampling the posterior distribution.

It builds a chain that has visited regions of parameter space with high posterior probability relatively more often than those with low posterior probability.

MH algorithm:

- 1) Start with initial (random) value of model parameters.
- 2) Choose new proposed parameters (random walk).
- 3) Accept new proposal for parameters based on the ratio of posteriors; i.e. draw random number between 0 and 1 and compare to ratio of posteriors
- 4) If accepted, store in chain, else, store the previous step, i.e. $\mathbf{X}^{i+1} = \mathbf{X}^i$ (**always storing**)
- 5) Repeat from 2

$$p_{\text{accept}} = \min \left[1, \frac{\frac{P(x|\mu)P(\mu)}{P(x)}}{\frac{P(x|\mu_0)P(\mu_0)}{P(x)}} = \frac{P(x|\mu)P(\mu)}{P(x|\mu_0)P(\mu_0)} \right]$$

MC Methods and Simulation

The introduced MC methods can be used to efficiently produce simulated data based on *prior* knowledge (eg. Standard Model parameters)

The MC model produces random events and accepts/rejects them based on the probability they are true

- In particle physics, this probability is the cross-section as a function of kinematics
- If probability density functions are chosen wisely, this should reproduce nature (modulo unknown or not-well described effects)

There are many tools available to produce particle physics simulations

- Eg. popular choice for collider physics is [MadGraph](#) (tutorial [here](#))

MH Example

Check out the [MetropolisHastings.ipynb](#) example, and try the exercises

Parameters of the MH algorithm:

- Step size: affects how the next example is drawn (can lead to autocorrelation)
- Burn in: the starting value takes a certain amount of time to be “forgotten” by the chain; this can lead to a bias in the posterior distribution.

The MH algorithm as shown cannot handle large datasets. Try to understand why and find out what can be done about it.

Underflow

When multiplying many small numbers, eventually one will get a number that is **too small** to be represented in memory using a floating point number.

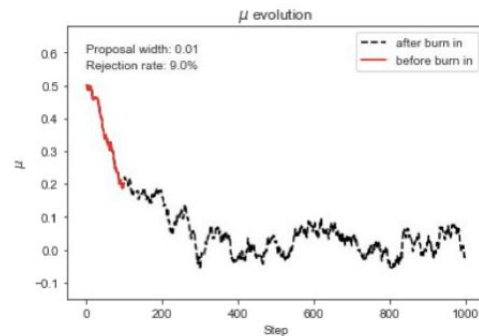
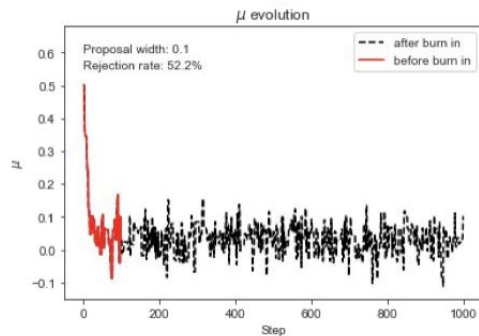
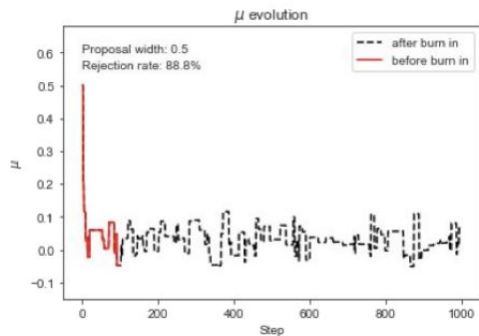
This means the more data points in the MH algorithm, the more likely it is to obtain an accept rate of 0 and reject (almost) all samples.

$$p(d|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \prod_i^N \exp\left(\frac{-(d_i - \mu)^2}{2\sigma^2}\right)$$

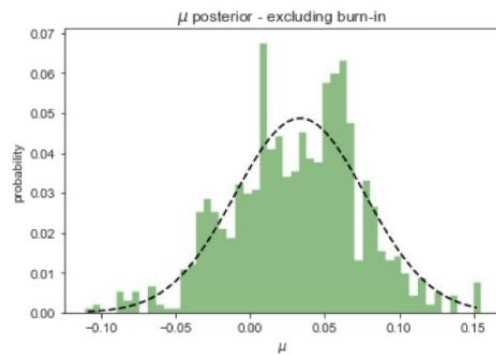
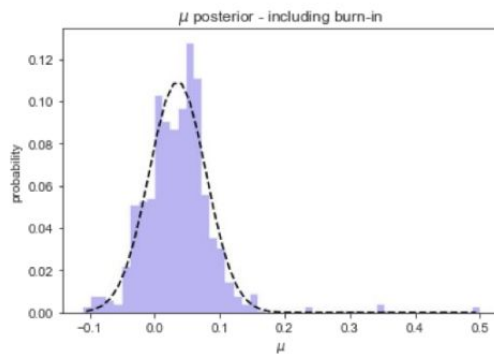
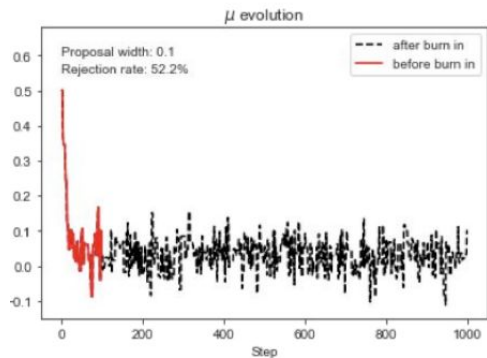
Solution: use negative of natural log of the likelihood, so that we only have to sum reasonable-sized negative numbers

MH Parameters

Step size



Burn in



PyMC3 is a python package for Bayesian modelling and machine learning.

- Does the MCMC & MH for you! Along with more advanced implementations!
- Can handle thousands of parameters
- Relies on Theano which dynamically compiles code before running MCMC.
- Has a suite of analysis tools and plots to optimise your MCMC.